

UNIVERSITÀ DEGLI STUDI DI ROMA TOR VERGATA

METODI DI OTTIMIZZAZIONE PER BIG DATA

Mars Operations: Base Deployment

Report del Progetto

Barbatano Enrico
Costantino Emanuele
Pera Cristiano

23 giugno 2026

Indice

1	Introduzione	2
2	Metodologia	2
2.1	Approssimazione di f_4	2
2.1.1	Analisi della topologia del problema	2
2.1.2	Ricerca Stocastica della baseline e generazione del dataset	4
2.1.3	Implementazione del modello	5
2.2	Minimizzazione di f	7
3	Analisi dei risultati	7
3.1	Qualità delle soluzioni	7
3.2	Convergenza	10
3.3	Limiti dell'approccio	11
4	Conclusioni	12

1 Introduzione

La traccia del progetto considera un'area marziana preventivamente individuata come potenzialmente idonea all'insediamento umano e richiede di determinare la configurazione spaziale ottimale di un insieme di 1000 moduli operativi distribuiti sul territorio.

La funzione obiettivo descritta nella traccia combina infatti diverse componenti matematiche che modellano aspetti differenti del sistema.

- f_1 : il costo associato alla dispersione
- f_2 : il costo associato alle distanze tra i moduli e le postazioni fisse
- f_3 : il costo associato alla deviazione dalla distanza operativa ideale tra ciascun modulo e il rispettivo punto di riferimento.
- f_4 : una black-box che modella la componente di costo aggiuntiva

L'obiettivo del presente progetto è sviluppare e analizzare un metodo di ottimizzazione capace di individuare configurazioni dei moduli che minimizzino il costo totale del sistema. A tal fine vengono studiate le caratteristiche matematiche del problema, viene progettata una strategia risolutiva adeguata alla sua elevata dimensionalità e vengono infine condotti esperimenti numerici per valutarne l'efficacia. Particolare attenzione è stata posta sulla funzione f_4 , essendo essa non nota e risultando la più onerosa.

Nelle sezioni successive verrà quindi analizzata la metodologia utilizzata per modellare il problema ed ottimizzarlo. Successivamente, verranno riportati e discussi i risultati ottenuti.

Il risultato trovato è presente nel file "x.txt", insieme al codice completo del programma creato per ricavarlo. È possibile trovare tutto il materiale nell'archivio MOBD.zip, reperibile al seguente [link](#), dove è presente anche il dataset utilizzato nel progetto, che verrà discusso ampiamente nelle sezioni successive.

2 Metodologia

L'intero progetto è stato realizzato tramite linguaggio Python, utilizzando la libreria NumPy. Tramite essa sono state implementate tutte le funzioni note (f_1 , f_2 e f_3), con i relativi gradienti. Per verificare la validità dei gradienti, è stata anche implementata una funzione che usa le differenze finite centrali. Tutte queste funzioni sono presenti nei file obj.py e gradients.py. In [appendice](#) vengono riportate tutte le formule matematiche implementate. Per quanto riguarda f_4 invece, non essendo nota, è stato utilizzato un approccio diverso.

Per l'ottimizzazione della funzione completa f , si è scelto di utilizzare il metodo del gradiente con momentum.

2.1 Approssimazione di f_4

Data l'impossibilità di ottenere una rappresentazione matematica di f_4 , si è deciso di implementare una rete neurale che simulasse questa funzione. In questo modo, è stato possibile utilizzare l'inferenza della NN nel calcolo della funzione totale f .

2.1.1 Analisi della topologia del problema

Come primo step esplorativo, si è proceduto alla minimizzazione della componente puramente analitica della funzione di costo, definita come $\tilde{f} = f_1 + f_2 + f_3$. Tale ottimizzazione è stata condotta avvalendosi dell'algoritmo L-BFGS, particolarmente indicato per la sua efficienza e

convergenza in problemi di ottimizzazione non lineare su larga scala. L'analisi spaziale della configurazione restituita come punto di minimo per $\tilde{f}$ ha rivelato una forte tendenza dei moduli ad allinearsi lungo la bisettrice principale (retta di equazione $x_1 - x_2 = 0$).

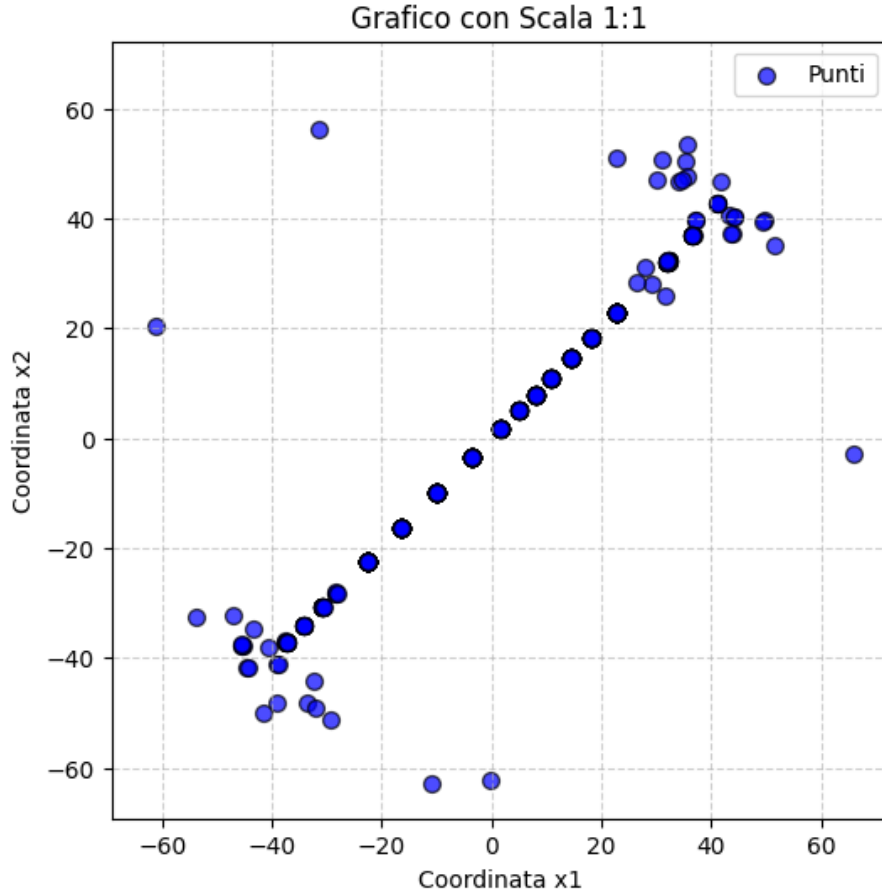


Figura 1: Configurazione minima di $f_1 + f_2 + f_3$ trovata con L-BFGS.

Tuttavia, è emerso un limite strutturale intrinseco a questa soluzione parziale: l'eccessiva prossimità spaziale tra i moduli, pur minimizzando le distanze operative (f_2, f_3), penalizza pesantemente la valutazione complessiva a causa del termine black-box f_4 . Per mitigare tale criticità, si è resa necessaria una dilatazione spaziale controllata della configurazione (si veda Figura 2). Attraverso una procedura dedicata (implementata nello script `espansione.py`), si è forzata la struttura originaria ad assumere una disposizione più sparsa, preservando l'allineamento lungo la retta $x_1 - x_2 = 0$ ma garantendo una maggiore separazione fisica. Nello specifico, tale sparsificazione è stata ottenuta allungando la configurazione iniziale moltiplicando i moduli per un fattore di dilatazione costante, sparsificando aggiungendo una matrice di rumore gaussiano e infine applicando una repulsione locale per evitare che i moduli siano troppo vicini fra loro.

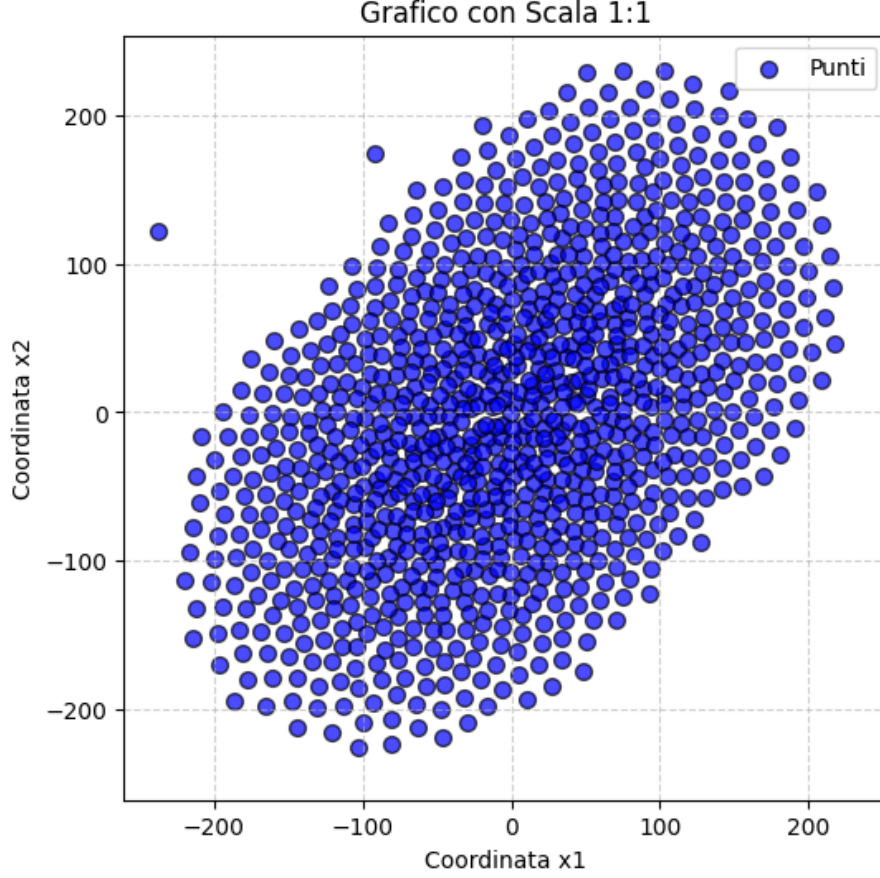


Figura 2: Dilatazione spaziale del punto di minimo di $f_1 + f_2 + f_3$.

2.1.2 Ricerca Stocastica della baseline e generazione del dataset

Ottenuta questa configurazione base sparsa, si è proceduto a una fase di esplorazione stocastica volta a individuare un posizionamento che garantisca un valore intrinsecamente basso del termine black-box f_4 . A tal fine, la configurazione è stata sottoposta a $n = 500$ perturbazioni gaussiane indipendenti. Per consentire un'esplorazione sufficientemente ampia del bacino di soluzioni, si è optato per un rumore gaussiano a media nulla e deviazione standard $\sigma = 20$. La configurazione che ha restituito il valore minimo di f_4 all'interno di questo campione esplorativo è stata scelta come baseline per la generazione del dataset definitivo.

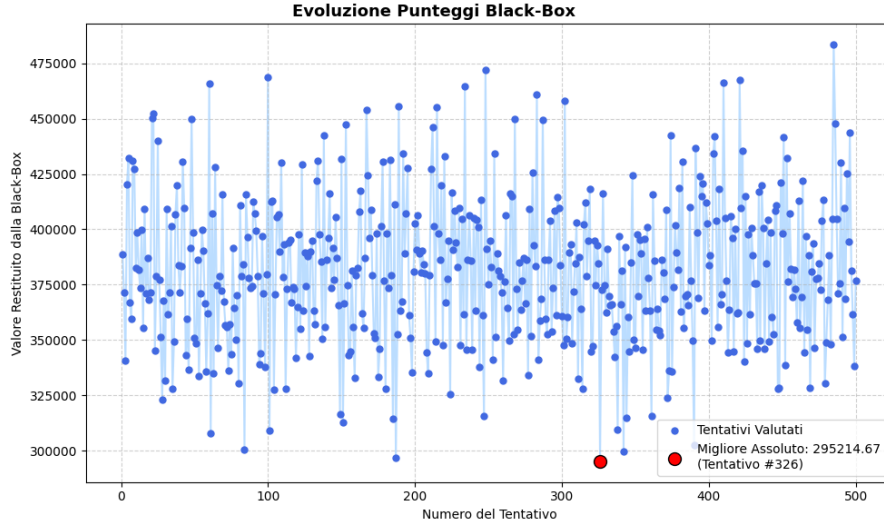


Figura 3: Ricerca della baseline.

L'ultima fase ha previsto un campionamento intensivo nell'intorno del punto di ancoraggio appena individuato, al fine di generare le 50.000 istanze necessarie per il training set. Per garantire un corretto bilanciamento tra l'esplorazione fine del minimo locale e la generalizzazione nelle aree adiacenti, le perturbazioni gaussiane (sempre a media nulla) applicate alla baseline sono state suddivise in due regimi di intensità:

- Il 40% dei campioni è stato generato applicando una deviazione standard $\sigma = 1$, con l'intento di campionare densamente l'immediato intorno della soluzione ottima nota.
- Il restante 60% dei campioni è stato generato con una deviazione standard $\sigma = 2$, promuovendo la scoperta di configurazioni limitrofe per fornire alla rete neurale una comprensione più ampia e generalizzabile della topologia locale della funzione obiettivo.

2.1.3 Implementazione del modello

Per l'implementazione del modello è stato utilizzato il framework **PyTorch**, che fornisce tutti gli strumenti necessari per implementare una semplice rete neurale. Tra le varie funzionalità offerte da questa libreria, è presente il calcolo del gradiente, che viene quindi effettuato in maniera automatica e restituito come una `tensor` NumPy. Questa funzionalità è stata fondamentale, in quanto ha permesso di integrare la NN nell'algoritmo di ottimizzazione scelto (**metodo del gradiente con momentum**).

1) Definizione del modello La rete neurale implementata è una rete *fully connected* e l'architettura è costituita da:

- un **input layer** di dimensione 2000;
- una sequenza di **hidden layers** di dimensione [1024, 516, 256, 128, 64, 32, 16];
- un **output layer** di dimensione 1.

La rete non utilizza i layer standard `nn.Linear`, ma layer definiti manualmente tramite la classe `LinearManual`, che implementa la trasformazione affine

$$x \mapsto xW + b,$$

dove W rappresenta la matrice dei pesi e b il vettore dei bias.

Per tutti i layer nascosti viene utilizzata la funzione di attivazione **GELU**, scelta perché più morbida della **ReLU**, capace di mantenere una buona continuità del gradiente e adatta a reti profonde di regressione. L'ultimo layer non applica alcuna attivazione, poiché l'output deve essere un valore continuo.

Per quanto riguarda la normalizzazione, ogni input reale x viene trasformato secondo la formula:

$$x_{\text{norm}} = \frac{x - \mu_x}{\sigma_x},$$

dove:

- μ_x è la media delle feature di input calcolata sul training set;
- σ_x è la deviazione standard delle feature di input calcolata sul training set.

In modo analogo, anche il target $y = f_4$ viene standardizzato:

$$y_{\text{norm}} = \frac{y - \mu_y}{\sigma_y},$$

dove:

- μ_y è la media dei valori di f_4 nel training set;
- σ_y è la deviazione standard dei valori di f_4 nel training set.

La standardizzazione degli input e del target consente di lavorare con grandezze numericamente più equilibrate, migliorando la stabilità e l'efficienza del processo di apprendimento.

2) Training L'addestramento è stato definito come un problema di regressione supervisionata. In particolare, il modello viene addestrato minimizzando una funzione di loss composta da:

1. la **Mean Squared Error (MSE)** tra predizione e target standardizzato:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2;$$

2. un termine di **regolarizzazione** L_2 sui parametri del modello:

$$\lambda \|\theta\|_2^2.$$

La loss totale è quindi:

$$\mathcal{L} = \mathcal{L}_{\text{MSE}} + \lambda \|\theta\|_2^2.$$

Per aggiornare i parametri è stato utilizzato l'ottimizzatore **CustomAdam**, implementazione manuale dell'algoritmo Adam, con learning rate pari a 10^{-3} , batch size pari a 64, massimo 300 epoche e *early stopping* con pazienza pari a 30 epoche. In questo modo il training si interrompe quando la loss di validazione non migliora per un numero sufficiente di epoche consecutive, evitando fenomeni di overfitting.

3) Inferenza Durante l'inferenza, l'input viene normalizzato con le stesse statistiche del training set. L'output della rete viene inizialmente prodotto nello spazio normalizzato e successivamente riportato nella scala reale mediante la denormalizzazione:

$$y_{\text{real}} = y_{\text{norm}} \cdot \sigma_y + \mu_y.$$

Questa operazione è necessaria perché, se da un lato il training beneficia di un target standardizzato, dall'altro in fase di utilizzo del modello è necessario ottenere una stima di f_4 nella sua scala reale, così da poterla combinare correttamente con le altre componenti della funzione obiettivo.

Infine, dopo la denormalizzazione, viene applicata la funzione **SoftPlus**:

$$\text{SoftPlus}(z) = \log(1 + e^z),$$

in modo da garantire che la predizione finale di f_4 sia non negativa. Questa scelta è coerente con la natura di f_4 , che nel problema viene interpretato come un costo e quindi non dovrebbe assumere valori negativi.

2.2 Minimizzazione di f

Per minimizzare f_4 , come già accennato, è stato usato il metodo **metodo del gradiente con momentum** perché, dopo un confronto, ha fornito migliori risultati rispetto a bfgs. L'algoritmo è stato implementato manualmente nel file `gdm.py`. Per poterlo utilizzare, è stato quindi realizzato uno script, chiamato `calculate_opt_total.py` che, a partire dalle implementazioni manuali dei gradienti delle funzioni note, e del gradiente della rete neurale, fornito da numpy, avvia l'ottimizzazione.

Nello specifico, questo script esegue l'ottimizzazione in un intorno ristretto di un punto di riferimento (x_{ref}), caricato da un file di testo e scelto randomicamente dal dataset, a seguito di alcune prove. La particolarità di questo codice risiede nel suo approccio alla limitazione dello spazio di ricerca: invece di ottimizzare direttamente le variabili originali x , definisce una nuova variabile non vincolata u . La trasformazione $z = x_{ref} + \Delta_{max} \cdot \tanh(u)$ garantisce che le perturbazioni rimangano confinate entro un raggio di Δ_{max} dal punto iniziale, applicando poi un clipping ai limiti di dominio stabiliti (-300, 300). Il wrapper `SurrogateObjective` gestisce queste conversioni e calcola i gradienti applicando la regola della catena (chain rule) per combinare i gradienti analitici rispetto a x con le derivate della trasformazione e con i gradienti di PyTorch ottenuti tramite `autograd`. Infine, l'ottimizzazione viene eseguita partendo da $u = 0$, per poi salvare su file sia la variabile ausiliaria ottima (u^*) che la soluzione ottima ricostruita nello spazio originale (x^*). È inoltre presente una forte configurazione per garantire la ripetibilità dei risultati numerici impostando seed fissi e forzando l'uso della CPU e di un singolo thread.

In questo modo è stato trovato il punto ottimale x^* , fornito come soluzione del problema. I risultati verranno discussi in maniera più esaustiva nella sezione successiva.

3 Analisi dei risultati

In questa sezione verranno analizzati i risultati ottenuti durante lo svolgimento del progetto. In particolare, verrà analizzata la qualità delle soluzioni e la convergenza della funzione obiettivo, fornendo grafici e discutendo i limiti dell'approccio utilizzato.

3.1 Qualità delle soluzioni

Per l'ottimizzazione finale, sono stati utilizzati due algoritmi: **L-BFGS** e **Metodo del Gradiente con Momentum**. I parametri utilizzati sono stati:

- Per **L-BFGS**: $\alpha = 1$, $\gamma = 10^{-4}$, $\Delta = 0.5$, $m = 5$
- Per **Metodo del Gradiente con Momentum**: $\alpha = 1$, $\beta = 0.3$

I punti migliori trovati sono stati molto simili, di seguito vengono riportati:

L-BFGS

- f : 465568.02
- f_1 : 49948.32
- f_2 : 155584.92
- f_3 : 14819.55
- f_4 : 245305.24

Metodo del Gradiente con Momentum

- f : 433566.79
- f_1 : 49948.44
- f_2 : 155500.09
- f_3 : 14991.34
- f_4 : 213126.92

Questi risultati confermano la superiorità, per questo problema, del **Metodo del Gradiente con Momentum**, in quanto i punti trovati ottengono risultati estremamente simili per quanto riguarda le funzioni note, ma mostrano notevoli miglioramenti sulla funzione black box, mostrando quindi maggiore compatibilità con la rete neurale implementata.

In entrambi i casi, notiamo che le soluzioni ottime sono distribuite lungo la diagonale principale, con forma ellittica. La densità dei punti è nettamente superiore nella zona centrale del grafico, in stretta prossimità dell'origine degli assi (0, 0). Man mano che ci si allontana dal centro verso la periferia dell'ellisse, i punti diventano gradualmente più radi. Questo è dovuto al fatto che la funzione f_2 dipende direttamente dalla norma euclidea dei moduli, costringendo la configurazione minima a non allontanarsi troppo dall'origine. La stragrande maggioranza dei dati ricade in un intervallo compreso approssimativamente tra -150 e 150 per entrambe le coordinate. I valori si estendono fino a toccare (e in rari casi superare leggermente) i limiti dei ± 200 . In generale non si notano valori palesemente anomali (outlier) completamente staccati dal gruppo principale. Anche i punti più periferici seguono la naturale e progressiva rarefazione della nuvola. Entrambi i grafici sono riportati nelle sezioni successive.

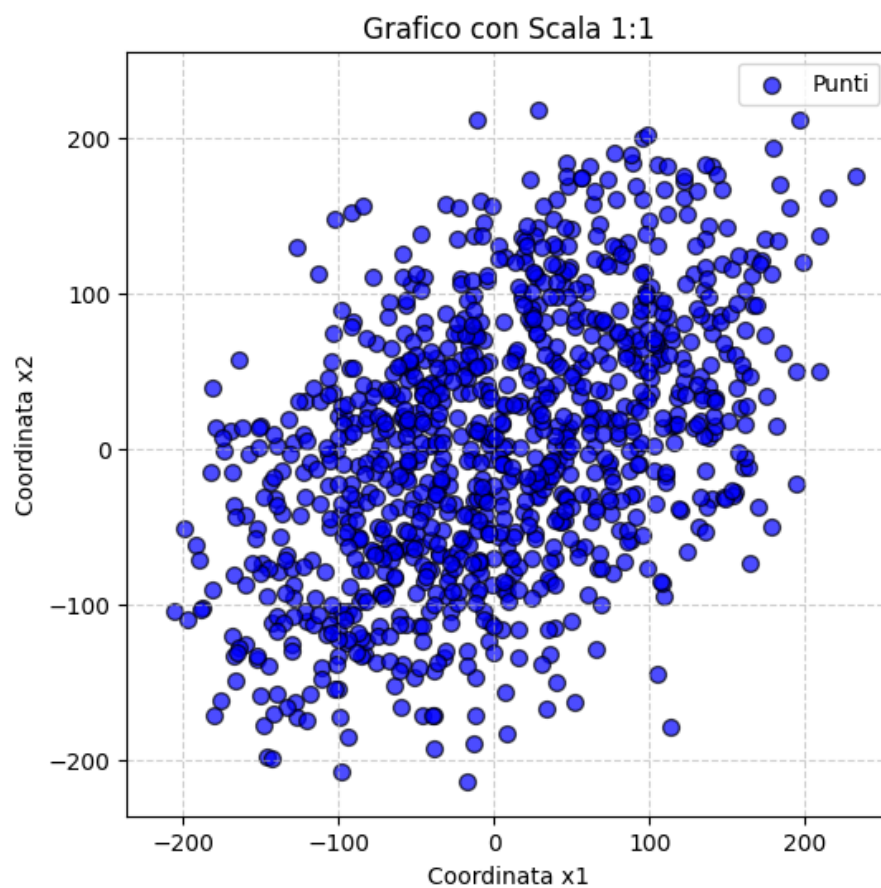


Figura 4: Posizionamento finale dei moduli con Gradiente Momentum.

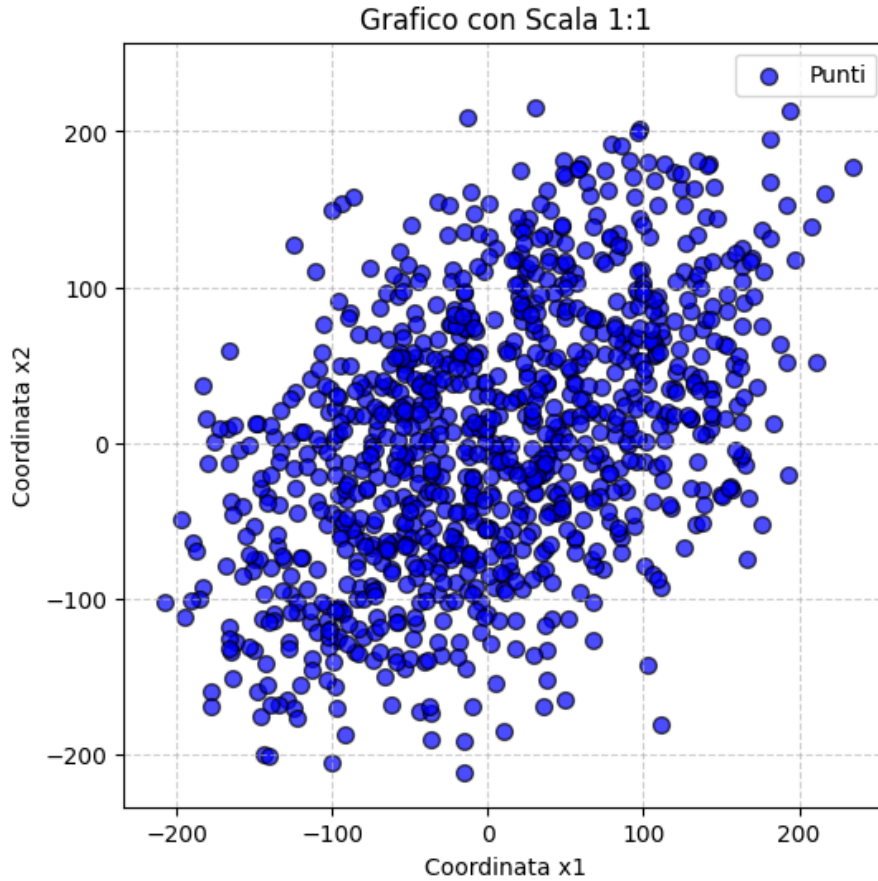


Figura 5: Posizionamento finale dei moduli con L-BFGS.

3.2 Convergenza

L'analisi della convergenza del metodo GDM evidenzia una topologia della funzione obiettivo altamente non lineare e malcondizionata. Il grafico della norma del gradiente (Figura 4) mostra una caduta quasi istantanea a valori prossimi allo zero nelle prime iterazioni. Il grafico della funzione obiettivo mostra un marcato andamento a gradini. Questo comportamento è indicativo della presenza di sub-ottimi locali, possibilmente causati dalle intense repulsioni a corto raggio tra i moduli. L'algoritmo è riuscito progressivamente a superare le congestioni locali (evidenziate dai salti discreti in Figura 5) fino a stabilizzarsi attorno all'iterazione 180.

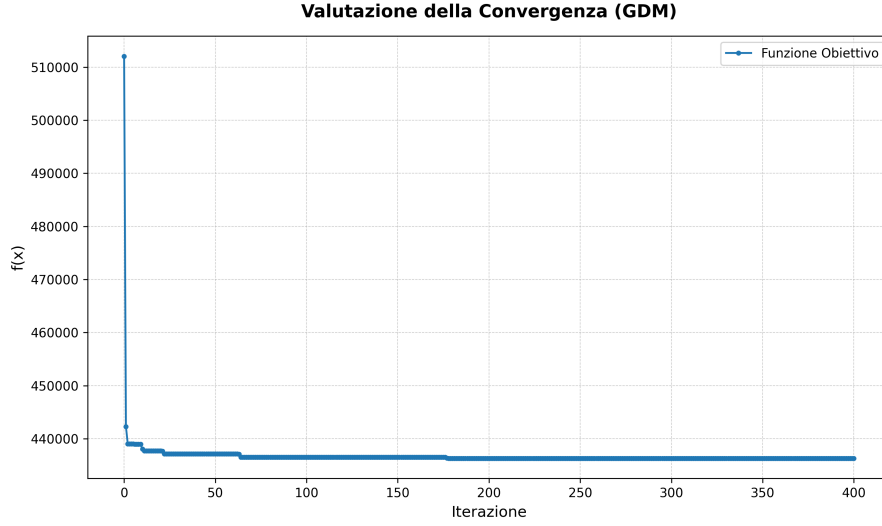


Figura 6: Andamento di f calcolata con la rete neurale.

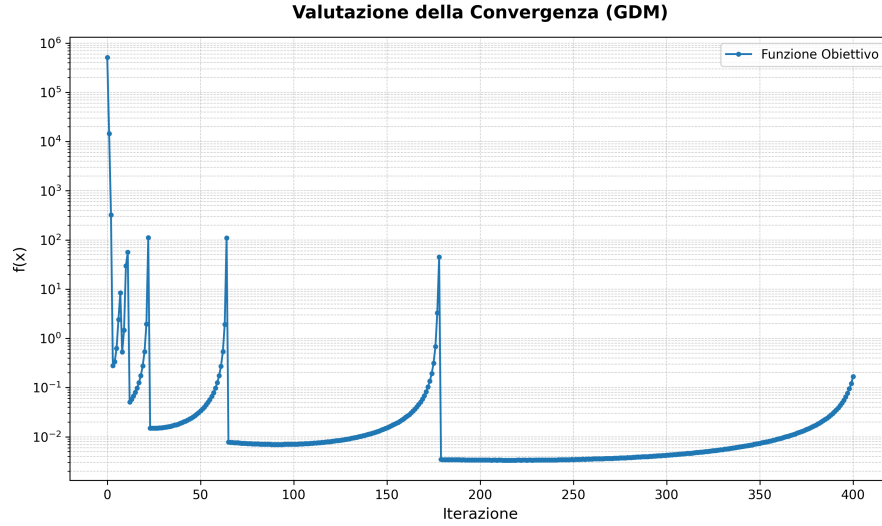


Figura 7: Andamento del gradiente di f calcolata con la rete neurale.

3.3 Limiti dell'approccio

Uno dei principali limiti dell'approccio adottato riguarda l'utilizzo di una rete neurale come *surrogato* della funzione f_4 . Infatti, il modello neurale non restituisce il valore reale di f_4 , ma una sua stima $\hat{f}_4$, appresa a partire dai dati disponibili durante la fase di training. Di conseguenza, l'ottimizzazione non viene eseguita sulla funzione reale completa, bensì sulla funzione approssimata

$$f_1(x) + f_2(x) + f_3(x) + \hat{f}_4(x).$$

Questo implica che la qualità della soluzione trovata dipende fortemente dall'accuratezza del modello neurale. Per questo motivo, una soluzione che appare ottima rispetto al surrogato non è necessariamente ottima anche rispetto al modello con la black box. In altre parole, il modello neurale rende il problema trattabile dal punto di vista computazionale, ma introduce inevitabilmente un margine di approssimazione che rappresenta il principale limite del metodo.

Inoltre la rete neurale non è addestrata per approssimare il valore della black box di una generica configurazione di moduli nello spazio, ma soltanto di punti prossimi al minimo di $f_1 + f_2 + f_3$. Questo implica che nonostante avendo trovato una configurazione minima per $f_1(x) + f_2(x) + f_3(x) + \hat{f}_4(x)$, è possibile che ne esista un'altra completamente diversa da quella trovata che rende minore il valore di f .

4 Conclusioni

In conclusione, il progetto ha permesso di affrontare in modo strutturato il problema del dispiegamento ottimale di una base marziana composta da 1000 moduli operativi, combinando strumenti di ottimizzazione numerica e apprendimento automatico. In particolare, è stato possibile trattare la componente black-box f_4 tramite un modello neurale surrogato, integrandolo all'interno della funzione obiettivo complessiva.

- **Obiettivi raggiunti:** sono state implementate correttamente le funzioni note del problema, f_1 , f_2 e f_3 , insieme ai rispettivi gradienti. È stata inoltre costruita una pipeline completa per l'approssimazione di f_4 , comprendente generazione del dataset, addestramento della rete neurale e successivo utilizzo del surrogate all'interno dell'algoritmo di ottimizzazione.
- **Risultati principali:** i risultati sperimentali mostrano che l'approccio proposto è in grado di produrre configurazioni spaziali plausibili e competitive. In particolare, il **Metodo del Gradiente con Momentum** ha fornito prestazioni migliori rispetto a L-BFGS nella minimizzazione della funzione totale approssimata, ottenendo una riduzione più significativa del contributo associato alla componente f_4 . Le soluzioni trovate mantengono inoltre una distribuzione geometrica coerente con le proprietà attese del problema.
- **Possibili miglioramenti futuri:** il principale limite dell'approccio adottato risiede nel fatto che l'ottimizzazione viene eseguita sulla funzione approssimata

$$f_1(x) + f_2(x) + f_3(x) + \hat{f}_4(x),$$

e non sulla funzione reale completa. Di conseguenza, possibili sviluppi futuri includono il miglioramento dell'accuratezza del surrogate neurale, l'ampliamento del dataset nelle regioni più promettenti dello spazio delle soluzioni e l'adozione di strategie di ottimizzazione ibride, in grado di combinare in modo più efficace esplorazione globale e raffinamento locale.

Appendice

Appendice A - Funzioni Matematiche

Parametri utilizzati

$$m = 1000, \quad r = 100$$

$$f_1(x^1, \dots, x^m) = 0.1 \sum_{i=1}^m \sum_{j>i}^m \left(1 - e^{-\|x^i - x^j\|}\right)$$

$$f_2(x^1, \dots, x^m) = \frac{1}{200} \sum_{i=1}^m \|x^i\|^2 + 80000$$

$$f_3(x^1, \dots, x^m) = 1000 \sum_{i=1}^m \left(\frac{\log(1 + \|x^i - s^i\|^2)}{\log(1 + r^2)} - 1 \right)^2$$

$$\nabla_{x^i} f_1(x^1, \dots, x^m) = 0.2 \sum_{i=2}^m e^{\|x^i - x^j\|^2} (x^i - x^j)$$

$$\nabla_{x^i} f_2(x^1, \dots, x^m) = \frac{1}{100} x^i$$

$$\nabla_{x^i} f_3(x^1, \dots, x^m) = \frac{4000}{\log(r^2 + 1)(1 + \|x^i - s^i\|^2)} \left(\frac{\log(1 + \|x^i - s^i\|^2)}{\log(r^2 + 1)} - 1 \right) (x^i - s^i)$$